

The background of the image features a dark blue gradient on the left, transitioning into a large, vibrant, abstract shape on the right. This shape is composed of overlapping curved segments in shades of orange, pink, and purple, creating a dynamic, modern aesthetic.

AWS re:Invent

NOV. 27 – DEC. 1, 2023 | LAS VEGAS, NV

SEC221

Centrally manage application secrets with AWS Secrets Manager

Ritesh Desai

(he/him)

GM, AWS Secrets Manager

AWS

Zach Miller

(he/him)

Sr. Security SA, Data Protection

AWS



“

**If you want to keep a secret,
you must also hide it from
yourself.**

George Orwell

1984



Agenda

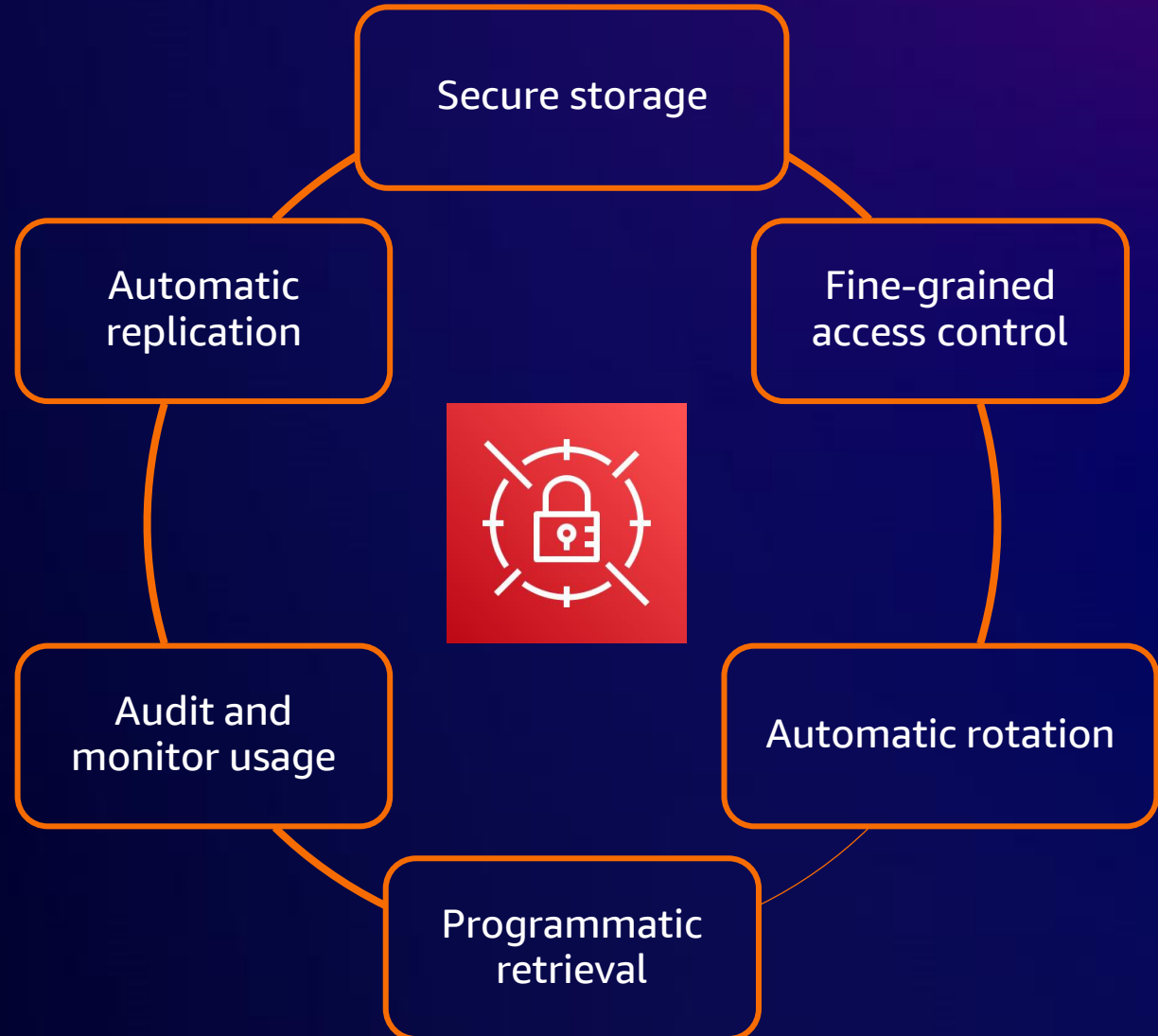
Introduction to AWS Secrets Manager

Whiteboarding + Q&A

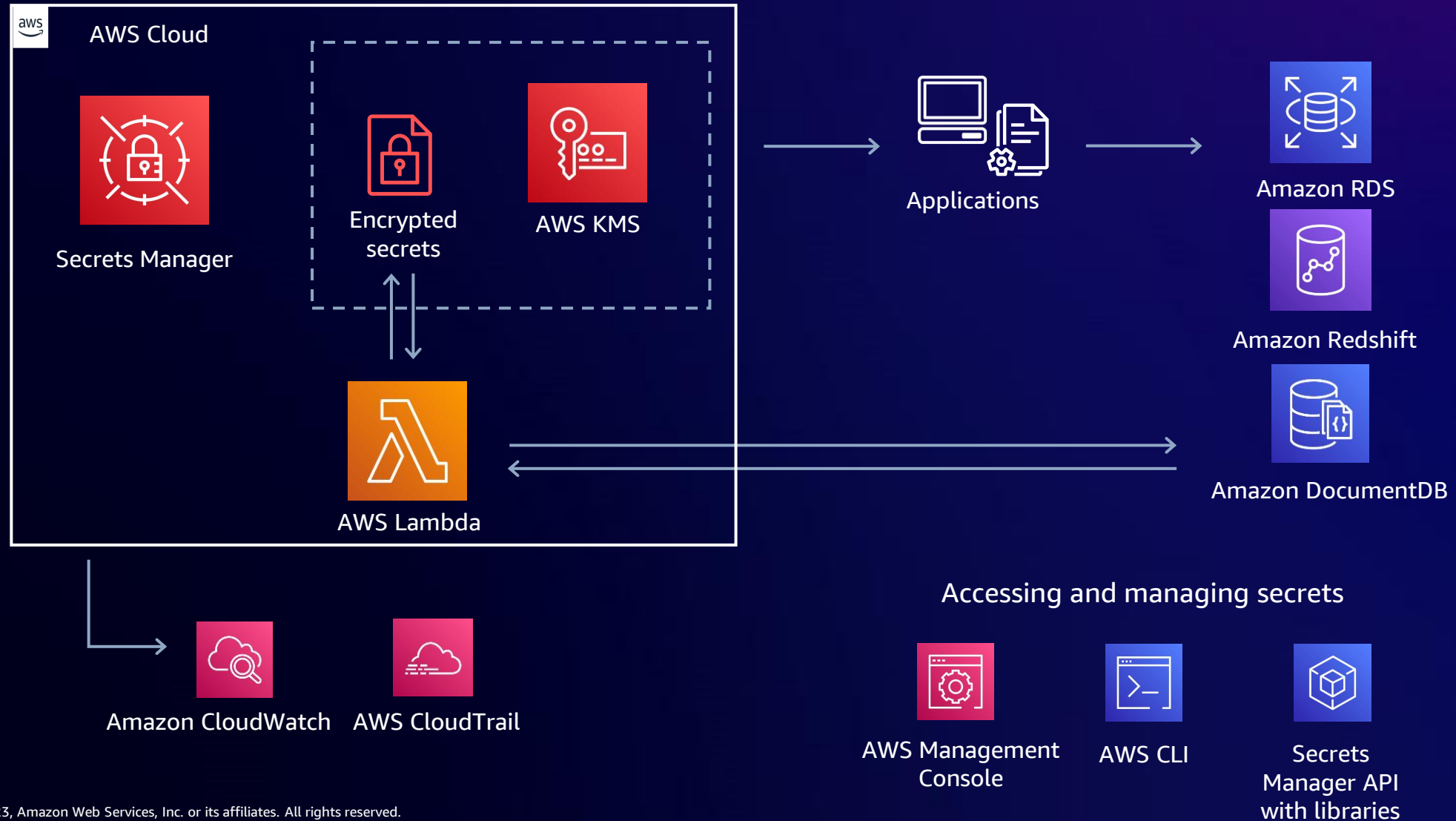
- **Creating secrets**
 - Fully managed admin secrets with Amazon RDS
 - Secrets Manager and AWS Systems Manager Parameter Store
- **Protecting secrets**
 - Access control patterns for secrets
 - Using AWS KMS with Secrets Manager secrets
- **Retrieving secrets**
 - Leverage existing integrations to consume secrets
 - Hybrid and multicloud access patterns

What is Secrets Manager?

Secrets Manager helps customers **manage, retrieve, and rotate** database credentials, API keys, and other **secrets** throughout their **lifecycles**



How Secrets Manager works



Creating secrets

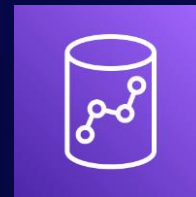
Fully managed database admin secrets

Overview

- You want to securely store and manage administrator credentials to your [Amazon RDS](#) and [Amazon Redshift](#) databases with as little operational overhead as possible
- When you create or modify your database, you can allow the database service to manage the admin password in Secrets Manager, including automatic secret rotation
- Database service generates the password and stores it in Secrets Manager

Benefits

- You do not need to configure or manage AWS Lambdas for secret rotation
- Password is generated by the relevant database service (e.g., Amazon RDS) and is never visible to your administrators
- You are in full control of access permissions and encryption key configuration, like any other secret in Secrets Manager



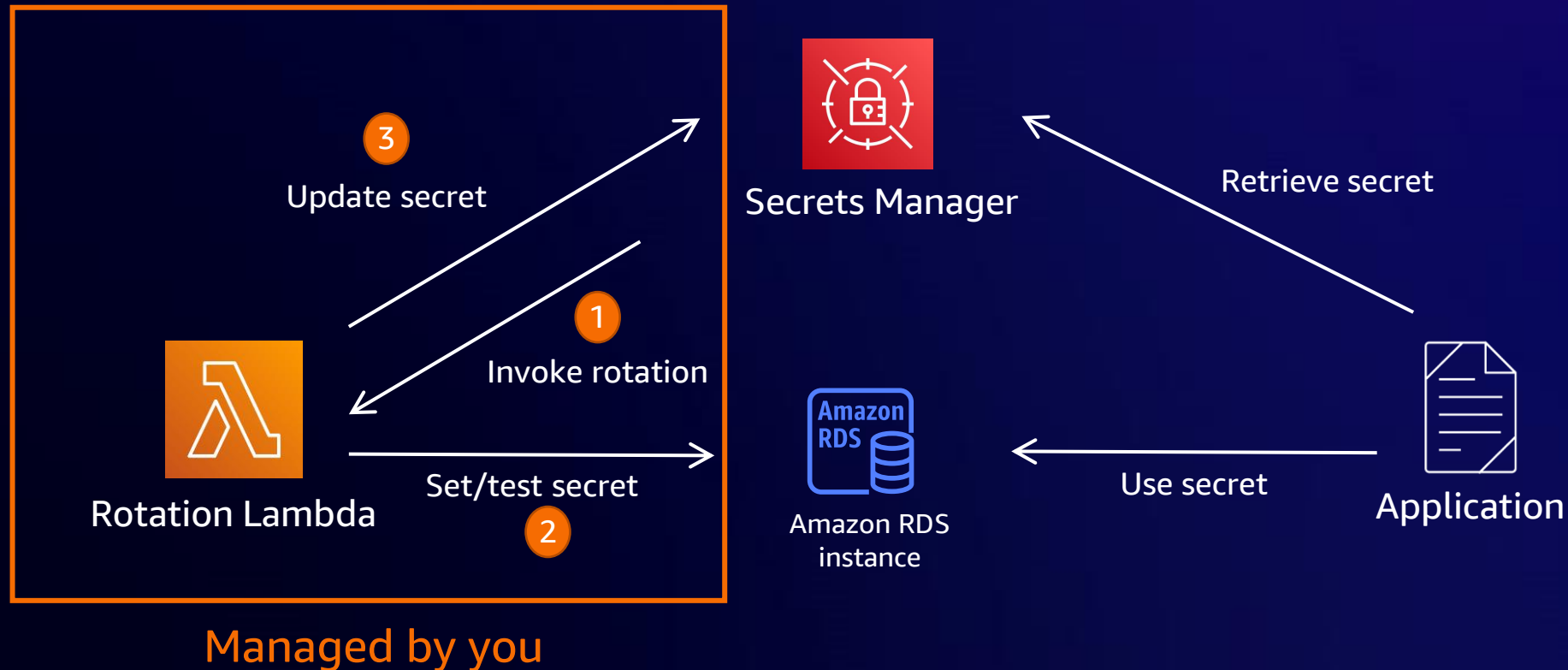
Amazon Redshift



Amazon RDS

Secret rotation managed by you

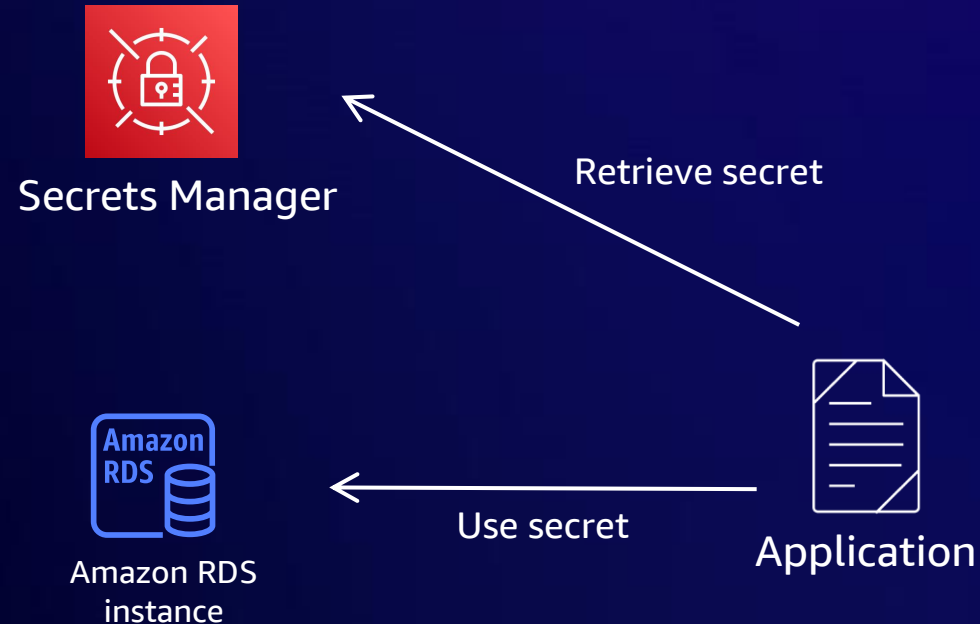
For secrets that you create, you are responsible for managing rotation Lambdas



Secret rotation managed by AWS

When you enable admin password management with Secrets Manager, AWS manages rotation on your behalf

Rotation managed by
AWS



Secrets Manager and AWS Systems Manager Parameter Store

Systems Manager Parameter Store is a capability that allows customers to store configuration management data. While Parameter Store does allow for encryption of sensitive parameters, Secrets Manager offers several features that make it the preferred choice for managing access to secrets.

Parameter Store benefits

- Low cost – “standard” parameters are **free**
- Provides a scalable, auditable repository for configuration data and database strings that are **not considered sensitive**



Systems Manager Parameter Store

Secrets Manager benefits

- Automatic secret rotation (for supported services)
- Secrets **encrypted by default**
- Client-side caching libraries **can improve performance** and **reduce cost** during secret retrieval
- Random secret generation
- **Minimum recovery period** ensures secrets cannot be accidentally deleted and lost

Secrets Manager and AWS Systems Manager Parameter Store

Secrets Manager is preferred when securely storing and accessing **sensitive data**, such as database credentials or application secrets. Secrets Manager Parameter Store should primarily be used for **non-secret configuration data**, such as license information or database connection strings.

Parameter Store use cases

- Database strings
- Amazon AMI IDs
- License codes

Secrets Manager use cases

- Database credentials
- Login information (username and password)
- Application secrets (Lambda, Amazon EC2, Amazon ECS/Amazon EKS)
- Third-party API keys or authentication tokens
- SSH keys



Secrets Manager

Protecting secrets

Secrets Manager access control policy types

Identity-based policies	Resource policy on secret	Resource policy on KMS key
1 to many	Many to 1	Many to 1
Provide access to multiple secrets for a single IAM entity (e.g., role)	Provide access to a single secret for multiple IAM entities (e.g., roles)	Provide access to a single KMS key for multiple IAM entities (e.g., roles)
Control permissions to API actions for resources that don't yet exist, or grant permissions to create new resources	Only controls permissions to an existing secret – the secret to which the resource policy is attached	Only controls permissions to an existing key – the secret to which the resource policy is attached
Grant access to users, roles, or groups in the same account	Grant access to users or roles in other AWS accounts	Grant access to users or roles in other AWS accounts
Required	Optional	Required

Scaling access using tags (ABAC)

- For **rapid growth environments**
- Simplifies management of access control pattern
- Create 1 or a small number of policies to **scale out access** to N number of secrets
- Strict **tagging enforcement required**

```
{
  "Version" : "2012-10-17",
  "Statement" : [ {
    "Effect": "Allow",
    "Principal" : {"AWS": "123456789012"},
    "Condition" : {
      "StringEquals" : {
        "aws:ResourceTag/access-project": "${aws:PrincipalTag/access-project}"
      }
    },
    "Action" : "secretsmanager:GetSecretValue",
    "Resource" : "*"
  } ]
}
```

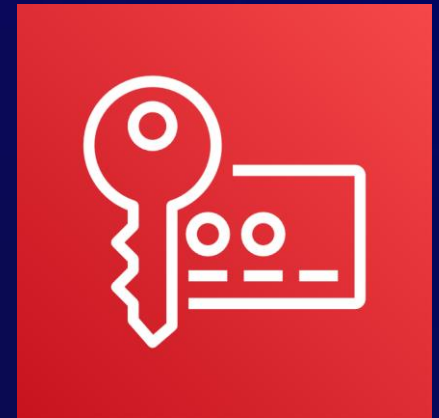
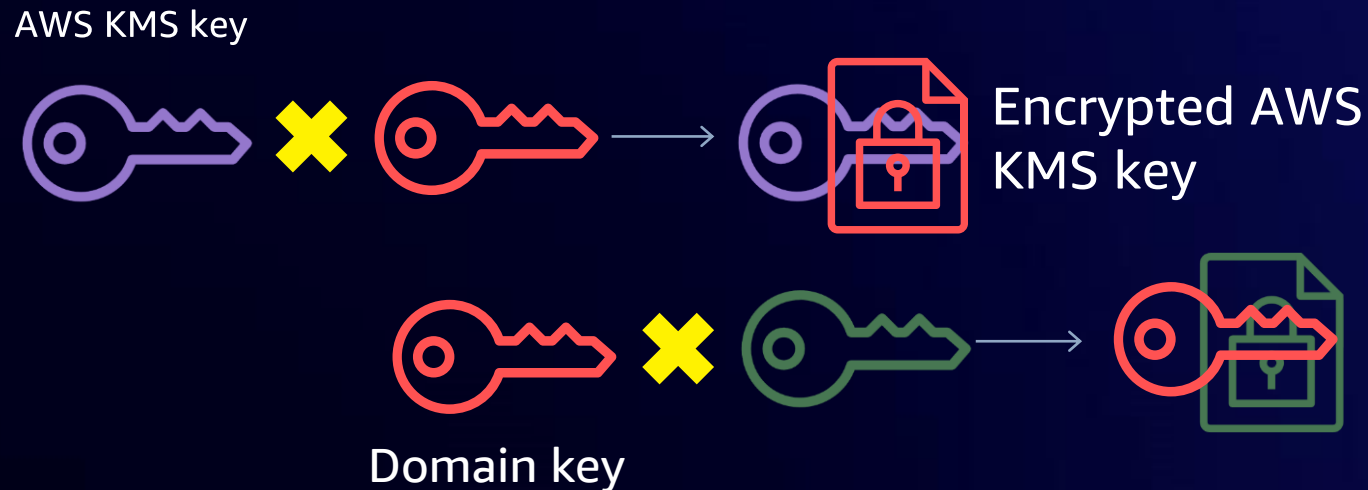
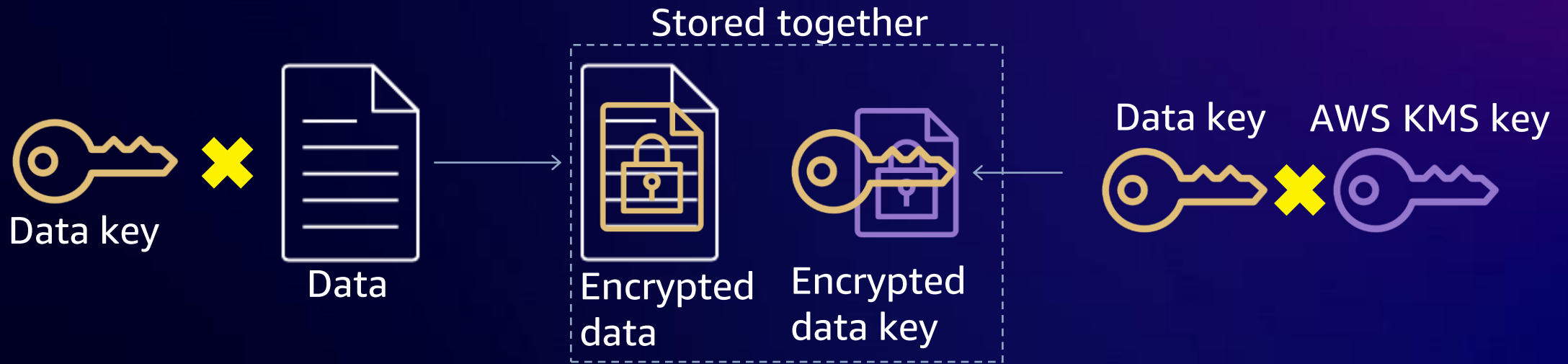
Example policy to manage secrets using tags

```
{  
  "Effect": "Allow",  
  "Action": [ "secretsmanager:GetResourcePolicy",  
    "secretsmanager:GetSecretValue",  
    "secretsmanager:DescribeSecret",  
    "secretsmanager:RestoreSecret",  
    "secretsmanager:PutSecretValue",  
    "secretsmanager:UpdateSecretVersionStage",  
    "secretsmanager>DeleteSecret",  
    "secretsmanager:ListSecretVersionIds",  
    "secretsmanager:UpdateSecret" ],  
  "Resource": "*",  
  "Condition": {  
    "StringEquals": {  
      "secretsmanager:ResourceTag/project": "${aws:PrincipalTag/project}"  
    }  
  }  
}
```

Only allow API actions
if these tags match



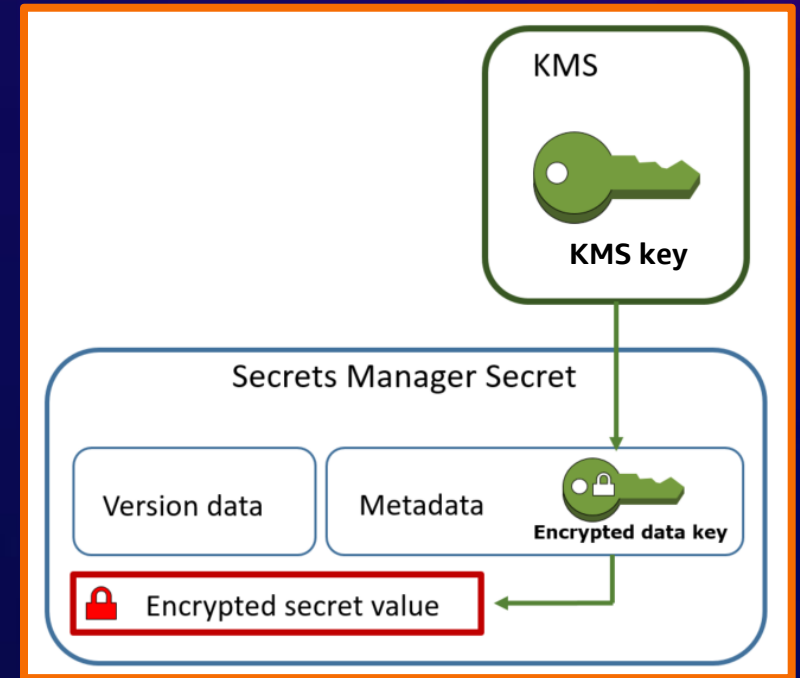
Envelope encryption primer



AWS KMS

How encrypting secrets works

- Encryption of secrets is **enabled by default** and **cannot be disabled**
- Secrets are encrypted with a **data key**, and that data key is encrypted with an AWS KMS key
- Encrypted data key is stored along with secret (in metadata)
- Secrets are **scrubbed from memory** after encryption and are **not** saved, unencrypted, in durable storage

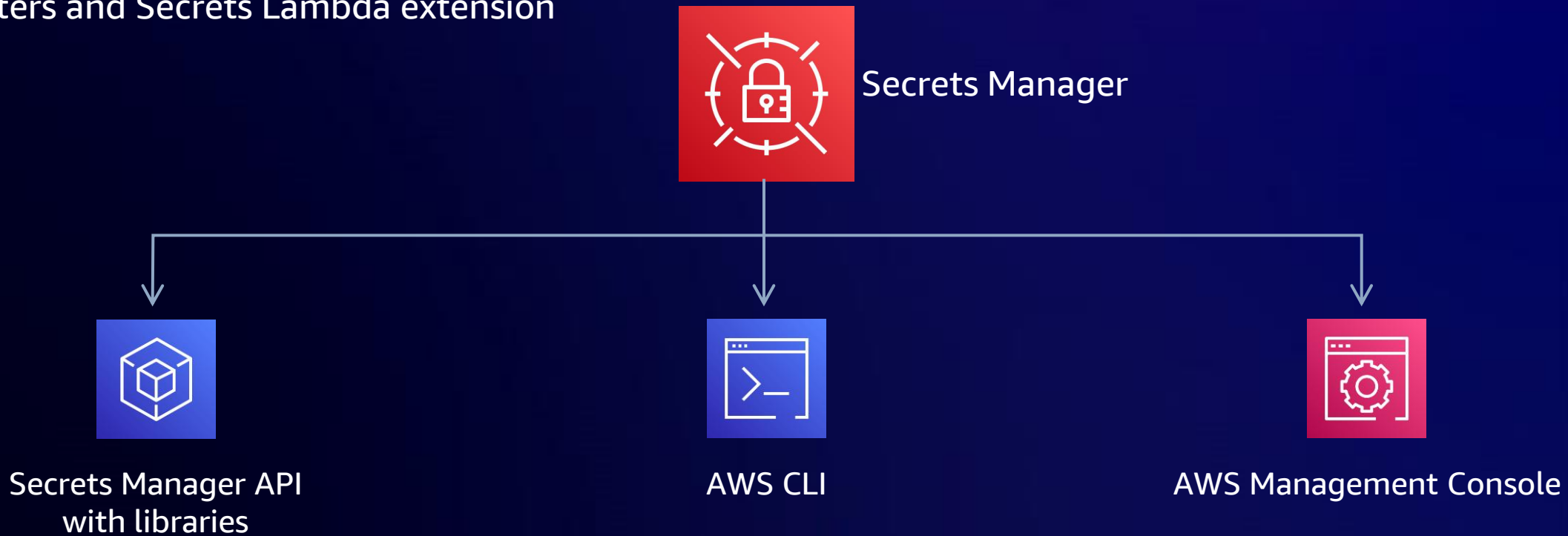


Retrieving secrets

Leverage existing integrations to consume secrets

Specialized integrations for secret retrieval

- Kubernetes: AWS Secrets and Configuration Provider
- Amazon ECS: Native integration
- AWS Parameters and Secrets Lambda extension



Hybrid and multicloud access patterns

Overview

- You want to use AWS to store and manage database credentials that can be accessed by resources **in AWS and on-premises devices**, such as application servers or Kubernetes clusters
- You can use the **same IAM roles and policies** you have configured for your AWS workloads to provide access to AWS resources

Benefits

- **Eliminate use of long-lived credentials** to access AWS services such as Secrets Manager
- Improve integration of hybrid workloads by standardizing on AWS security services
- Migrate from products with expensive licensing fees in favor of pay-as-you-go services from AWS

IAM Roles Anywhere

EXTENDS THE USE OF IAM ROLES TO WORKLOADS OUTSIDE OF AWS

IAM Roles Anywhere



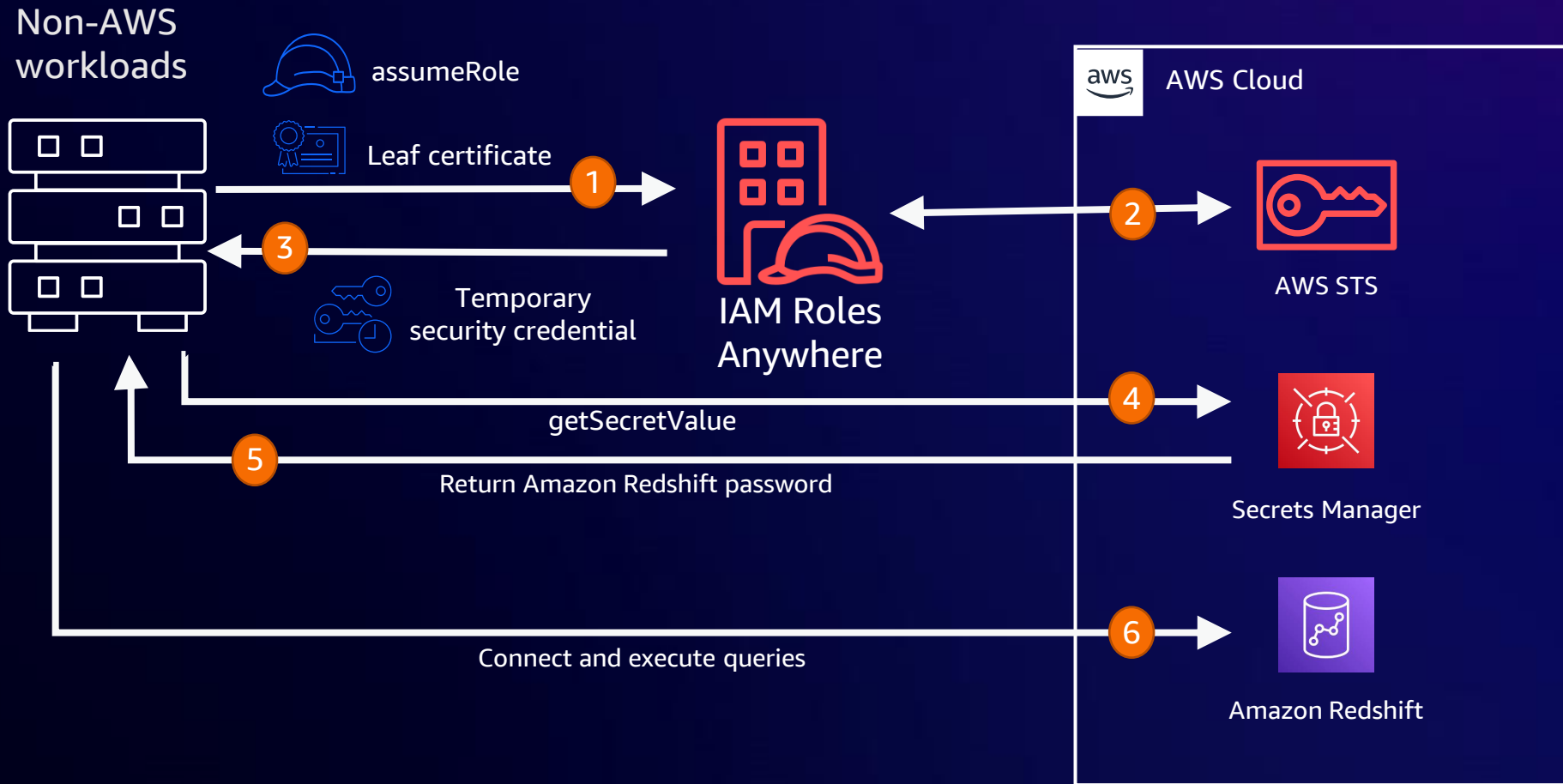
Uses the same IAM roles and policies you have already configured for your AWS workloads to access AWS resources on premises

Obtain temporary AWS credentials and eliminate long-lived credentials

Use the same access controls, deployment pipelines, and testing processes across all your workloads

Simplify the migration of your workloads running outside of AWS

Hybrid workloads use case



New capability to retrieve secrets in a group

Overview

- You want to use AWS to retrieve multiple secrets at once, as part of a group of related secrets
- You can now use AWS Secrets Manager to group related secrets and retrieve them with a single API call



AWS Secrets Manager

Benefits

- Optimize application workloads by streamlining the process to identify and retrieve a group of secrets
- Improve performance by decreasing the number of API calls, leading to reduced costs
- Enforce the same level of security, with the same access control policies in place for each individual secret

Questions?



Recap

- Secrets Manager provides a fully managed AWS service to create, protect, and retrieve application secrets
- Use **identity**, **resource**, and AWS KMS **key policies** to enable least-privilege access to application secrets
- **Retrieve secrets** from the AWS Management Console, directly from your applications, or using existing integrations with compute services, such as Amazon ECS

Thank you!



Please complete the session survey in the mobile app

Ritesh Desai



ritedesa@amazon.com



[linkedin.com/in/ritesh-desai](https://www.linkedin.com/in/ritesh-desai)

Zach Miller



zachml@amazon.com

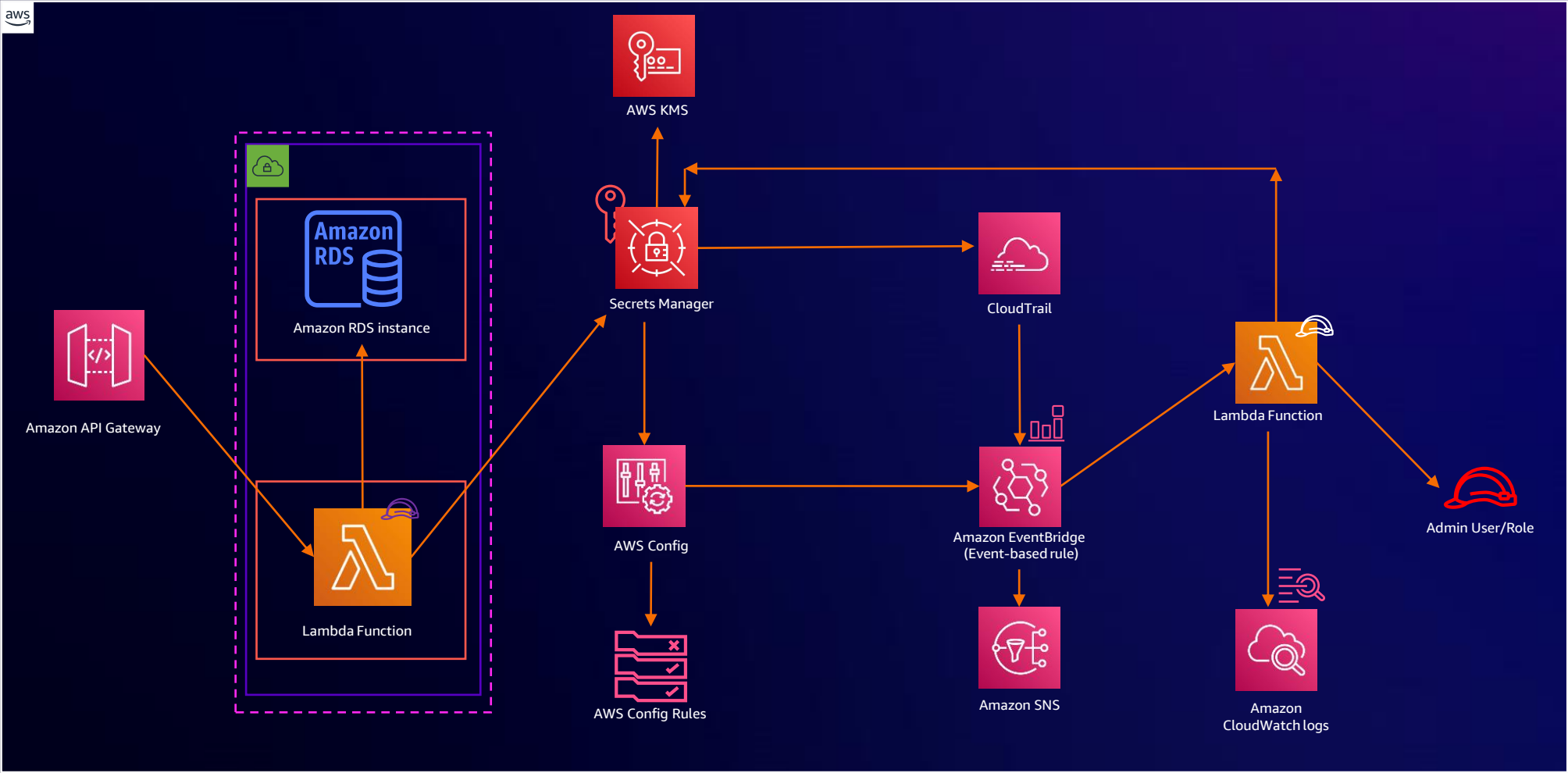


[linkedin.com/in/zmiller965](https://www.linkedin.com/in/zmiller965)



Appendix

Reference architecture



Example questions

- How do I retrieve secrets from an on-premises web server?
- How do I retrieve secrets from a workload in another cloud provider?
- How do I choose between Secrets Manager and Systems Manager Parameter Store
- Does AWS KMS encrypt my secrets? Does AWS CloudHSM encrypt my secrets?
- Can I retrieve many secrets at once?
- What if I don't want to manage rotation lambdas?
- What if my application calls Secrets Manager while a secret is being rotated?

Cross-account access: How to set policy

Use a resource-based policy attached to a secret to **give access to an IAM principal** in a different AWS account

Example 1: Specify only the other account ID

```
"Principal": {"AWS": "arn:aws:iam::AccountId:root"}
```

Example 2: Specify the **exact user or role in the other account**

```
"Principal": [  
  {"AWS": "arn:aws:iam::AccountId:role/MyCrossAccountRole"},  
  {"AWS": "arn:aws:iam::AccountId:user/MyUserName"}  
]
```

Scaling access using tags

- For **rapid growth environments**
- Simplifies management of access control pattern
- Create 1 or a small number of policies to **scale out access** to N number of secrets
- Strict **tagging enforcement** required

```
{
  "Version" : "2012-10-17",
  "Statement" : [ {
    "Effect": "Allow",
    "Principal" : {"AWS": "123456789012"},
    "Condition" : {
      "StringEquals" : {
        "aws:ResourceTag/access-project": "${aws:PrincipalTag/access-project}"
      }
    },
    "Action" : "secretsmanager:GetSecretValue",
    "Resource" : "*"
  } ]
}
```

Permission policy to require attributes on new secrets

```
{
  "Version": "2012-10-17",
  "Statement": [ {
    "Effect": "Allow",
    "Action": [
      "secretsmanager:CreateSecret" ],
    "Resource": "*",
    "Condition": {
      "StringEquals": {
        "aws:RequestTag/project": "${aws:PrincipalTag/project}",
        "aws:RequestTag/costcenter": "${aws:PrincipalTag/costcenter}" },
      "ForAllValues:StringEquals": {
        "aws:TagKeys": [
          "project",
          "createdBy",
          "costcenter",
          "application" ] } } } ] }
```

Requires **project** and **costcenter** tags and must be this value

Allows project with these keys, but nothing else

Permission policy to manage secrets using tags

```
{
  "Effect": "Allow",
  "Action": [ "secretsmanager:GetResourcePolicy",
    "secretsmanager:GetSecretValue",
    "secretsmanager:DescribeSecret",
    "secretsmanager:RestoreSecret",
    "secretsmanager:PutSecretValue",
    "secretsmanager:UpdateSecretVersionStage",
    "secretsmanager>DeleteSecret",
    "secretsmanager:RotateSecret",
    "secretsmanager:CancelRotateSecret",
    "secretsmanager:ListSecretVersionIds",
    "secretsmanager:UpdateSecret" ],
  "Resource": "*",
  "Condition": {
    "StringEquals": {
      "secretsmanager:ResourceTag/project": "${aws:PrincipalTag/project}"
    }
  }
}
```

Only manage resources
with these tags

Permission policy to manage secrets using tags

```
"Effect": "Allow",
"Action": ["secretsmanager:TagResource"],
"Resource": "*",
"Condition": {
  "StringEquals": {
    "secretsmanager:ResourceTag/project": "${aws:PrincipalTag/project}",
  },
  "ForAllValues:StringEquals": {
    "aws:TagKeys": [
      "project",
      "createdBy",
      "application",
      "costcenter" ]
  },
  "StringEqualsIfExists": {
    "aws:RequestTag/project": ["${aws:PrincipalTag/project}"],
    "aws:RequestTag/costcenter": [ "${aws:PrincipalTag/costcenter}" ] }
  }
}
```

Only tag resources with these tags

Tag with either of these keys

For **project**, you specify only these values

Permission policy to manage secrets using tags

```
"Effect": "Allow",
"Action": [
    "secretsmanager:UntagResource" ],
"Resource": "*",
"Condition": {
    "StringEquals": {
        "secretsmanager:ResourceTag/project": "${aws:PrincipalTag/project}" },
    "ForAllValues:StringEquals": {
        "aws:TagKeys": [
            "application" ] } } } }
```

Only change tags of
your project

AWS Parameter and Secrets Lambda extension

Overview and benefits

- You can retrieve and locally cache parameters from Systems Manager Parameter Store and secrets from Secrets Manager
- Extension can be added to Lambda functions by attaching an additional lambda layer to that function
- Reduces latency and cost in retrieving parameters and secrets

